TỐI ƯU HÓA MÃ HÓA SAT CHO BÀI TOÁN N-QUEENS VÀ ĐÁNH GIÁ SO SÁNH HIỆU NĂNG VỚI CÁC BỘ GIẢI CÔNG NGHIỆP

Nguyễn Đức Mạnh

Khoa Công nghệ Thông tin, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội
144 Xuân Thủy, Cầu Giấy, Hà Nội, Việt Nam

THÔNG TIN BÁO CÁO

Từ khóa:
N-Queens, SAT Solving, PySAT, At-Most-One Encoding, Integer Linear Programming (ILP), Constraint Programming (CP), CDCL.

TÓM TẮT

Bài toán N-Queens là bài toán thỏa mãn ràng buộc (CSP) kinh điển trong Khoa học Máy tính. Báo cáo này trình bày giải pháp tối ưu hóa việc giải bài toán N-Queens bằng cách chuyển đổi sang bài toán Thỏa mãn Công thức Logic Mệnh đề (SAT) thông qua thư viện `PySAT`. Chúng tôi phân tích chuyên sâu và đánh giá thực nghiệm 5 kỹ thuật mã hóa At-Most-One (AMO): Pairwise/Binomial, Sequential Counter, Binary, Commander và Product Encoding. Đồng thời, báo cáo tiến hành thử nghiệm so sánh trực diện hiệu năng giữa bộ giải SAT (CaDiCaL, Glucose) với các công cụ giải công nghiệp hàng đầu thuộc trường phái Lập trình Tuyến tính Nguyên (ILP: Gurobi, CPLEX MIP) và Lập trình Ràng buộc (CP: CPLEX CP, Google OR-Tools CP-SAT) trên bàn cờ kích thước lên tới $N = 1000$. Kết quả cho thấy việc chọn kỹ thuật mã hóa AMO phù hợp (đặc biệt là Product Encoding) triệt tiêu hiệu quả hiện tượng bùng nổ mệnh đề, giúp SAT Solvers vượt qua các hệ giải ILP thương mại và đạt hiệu năng vượt trội ở quy mô lớn.

1 Mở đầu (Introduction)

1.1 Đặt bài toán và Động lực nghiên cứu

Bài toán N-Queens đòi hỏi việc đặt N quân hậu trên một bàn cờ kích thước $N \times N$ sao cho không có hai quân hậu nào có thể tấn công nhau. Quy tắc này tương ứng với 3 điều kiện khắt khe: không có hai quân hậu nào nằm trên cùng một hàng, cùng một cột, hoặc trên cùng một đường chéo (chéo xuôi và chéo ngược).

Bài toán N-Queens không chỉ mang tính lý thuyết tổ hợp mà còn có ứng dụng quan trọng trong nhiều lĩnh vực kỹ thuật:

- Lập lịch tác vụ và phân bổ tài nguyên hệ thống (Register Allocation, Deadlock Prevention).

- Thiết kế vi mạch tích hợp quy mô cực lớn (VLSI Circuit Design).
- Điều hành giao thông đường không và tối ưu hóa đường đi (Traveling Salesman Problem).

1.2 Nhược điểm của các phương pháp truyền thống

Các thuật toán truyền thống như Quay lùi (Backtracking) và Nhánh và Cận (Branch and Bound) hoạt động dựa trên việc duyệt cây không gian trạng thái. Khi kích thước N tăng lên, số lượng cấu hình gia tăng theo hàm giai thừa ($O(N!)$), khiến các thuật toán này bị bùng nổ không gian tìm kiếm và không thể giải quyết trong thời gian chấp nhận được đối với $N \geq 30$.

1.3 Phương pháp tiếp cận SAT Solving

Việc chuyển đổi bài toán N-Queens sang bài toán Thỏa mãn Công thức Logic Mệnh đề (Boolean Satisfiability – SAT) và sử dụng các SAT Solvers hiện đại dựa trên thuật toán CDCL (Conflict-Driven Clause Learning) mở ra khả năng xử lý bài toán ở quy mô lớn. Tuy nhiên, cách mã hóa thông thường (Pairwise/Binomial Encoding) sinh ra $O(N^3)$ mệnh đề, dẫn đến hiện tượng bùng nổ mệnh đề (Clause Explosion) và tràn bộ nhớ (OOM) khi N lớn.

Đóng góp chính của báo cáo bao gồm:

1. Biểu diễn và cài đặt hoàn chỉnh mô hình SAT CNF cho bài toán N-Queens dựa trên thư viện `PySAT`.
2. Cài đặt, phân tích và so sánh thực nghiệm 5 kỹ thuật mã hóa At-Most-One (AMO): Pairwise/Binomial, Sequential Counter, Binary, Commander, và Product Encoding.
3. Đánh giá hiệu năng đối đầu trực tiếp giữa SAT Solvers với các công cụ giải ILP (Gurobi, CPLEX MIP) và CP (CPLEX CP, Google OR-Tools CP-SAT) trên ma trận kích thước N từ 4 đến 1000.

2 Mô hình hóa Toán học (Mathematical Formulations)

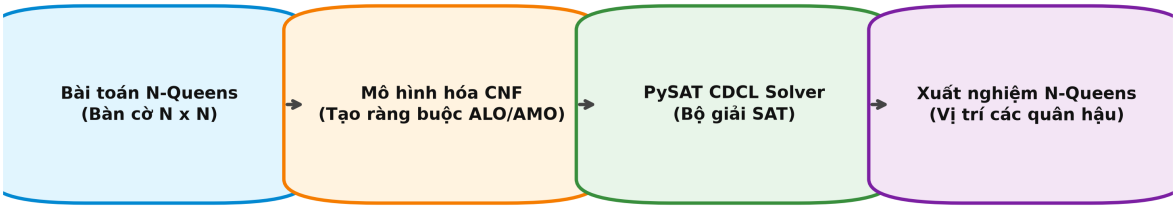
2.1 Biểu diễn Biến nhị phân (Boolean Variables)

Gọi $x_{i,j} \in \{0, 1\}$ là biến mệnh đề nhị phân đại diện cho ô tại hàng i, cột j ($1 \leq i, j \leq N$):

$$x_{i,j} = \begin{cases} 1, & \text{nếu đặt quân hậu tại vị trí } (i, j) \\ 0, & \text{ngược lại} \end{cases} \tag{1}$$

2.2 Mã hóa SAT CNF (Exact-One Constraint)

Điều kiện "Đúng một quân hậu" (Exact-One – EXO) trên mỗi hàng/cột được tách thành hai điều kiện sơ cấp: At-Least-One (ALO) và At-Most-One (AMO). Sơ đồ luồng xử lý bài toán được thể hiện trong Hình 1.



Hình 1: Sơ đồ luồng kiến trúc giải Bài toán N-Queens bằng phương pháp SAT Solving.

2.2.1 Ràng buộc At-Least-One (ALO)

Mỗi hàng và mỗi cột phải chứa ít nhất 1 quân hậu:

$$\bigwedge_{i=1}^{N} \left(\bigvee_{j=1}^{N} x_{i,j} \right) \quad \text{(Mỗi hàng có ít nhất 1 quân hậu)} \tag{2}$$

$$\bigwedge_{j=1}^{N} \left(\bigvee_{i=1}^{N} x_{i,j} \right) \quad \text{(Mỗi cột có ít nhất 1 quân hậu)} \tag{3}$$

2.2.2 Phân tích 5 Kỹ thuật Mã hóa At-Most-One (AMO)

Ràng buộc AMO yêu cầu tập biến $X = \{x_1, x_2, \ldots, x_m\}$ chứa tối đa một biến nhận giá trị 1:

1. **Pairwise / Binomial Encoding:**

$$\bigwedge_{1 \le i < j \le m} (\neg x_i \vee \neg x_j) \tag{4}$$

   Không sử dụng biến phụ trợ (0 biến phụ trợ), số lượng mệnh đề sinh ra là $O(m^2)$.

2. **Sequential Counter Encoding (Sinz, 2005):** Xây dựng mạch đếm lũy tiến sử dụng các biến phụ trợ s_i ($s_i = 1$ nếu có ít nhất 1 biến $x_k = 1$ với $k \le i$). Cần $m-1$ biến phụ trợ ($O(m)$ biến phụ), độ phức tạp số mệnh đề giảm xuống $O(m)$.

3. **Binary (Bitwise) Encoding:** Mã hóa chỉ số của biến thành chuỗi bit nhị phân độ dài $k = \lceil \log_2 m \rceil$. Cần $O(\log_2 m)$ biến phụ trợ và sinh ra $O(m \log_2 m)$ mệnh đề.

4. **Commander Encoding:** Chia m biến thành các nhóm nhỏ kích thước k, mỗi nhóm gán một biến chỉ huy (commander variable) c_i để quản lý trạng thái kích hoạt. Cần $O(m/k)$ biến phụ trợ.

5. **Product Encoding (Greshuk et al.):** Chiếu chỉ số $k \in \{1, \ldots, m\}$ của tập biến lên lưới ma trận 2 chiều kích thước $p \times q$ với $p = \lceil \sqrt{m} \rceil$ và $q = \lceil m/p \rceil$, tương ứng với tọa độ (u, v) trong đó $u \in \{1, \ldots, p\}$ và $v \in \{1, \ldots, q\}$. Tạo p biến phụ trợ hàng r_u và q biến phụ trợ cột c_v (tổng cộng $O(\sqrt{m})$ biến phụ trợ). Mệnh đề liên kết biến gốc x_k với biến phụ trợ được thiết lập qua quan hệ kéo theo $x_k \to (r_u \wedge c_v)$, tương đương với hai mệnh đề đơn:

$$(\neg x_k \vee r_u) \quad \text{và} \quad (\neg x_k \vee c_v) \tag{5}$$

   Sau đó, áp dụng mã hóa Pairwise trực tiếp lên tập p biến hàng $\{r_1, \ldots, r_p\}$ và q biến cột $\{c_1, \ldots, c_q\}$. Nhờ đó, số lượng biến phụ trợ chỉ ở mức $O(\sqrt{m})$ và số lượng mệnh đề

cho mỗi ràng buộc AMO giảm từ $O(m^2)$ (của Pairwise) xuống mức tuyến tính $O(m)$, triệt tiêu hiệu quả hiện tượng bùng nổ mệnh đề khi kích thước bàn cờ tăng lớn.

2.3 Mô hình Quy hoạch Tuyến tính Nguyên (ILP Formulation)

Mô hình biểu diễn cho các Solver ILP thương mại (Gurobi, CPLEX MIP):

$$\text{Minimize } 0 \quad \text{(Tìm giải pháp khả thi)} \tag{6}$$

$$\text{Subject to: } \sum_{j=1}^{N} x_{i,j} = 1, \quad \forall i \in \{1, \dots, N\} \quad \text{(Hàng)} \tag{7}$$

$$\sum_{i=1}^{N} x_{i,j} = 1, \quad \forall j \in \{1, \dots, N\} \quad \text{(Cột)} \tag{8}$$

$$\sum_{i+j=k} x_{i,j} \leq 1, \quad \forall k \in \{2, \dots, 2N\} \quad \text{(Chéo xuôi)} \tag{9}$$

$$\sum_{i-j=k} x_{i,j} \leq 1, \quad \forall k \in \{-(N-1), \dots, N-1\} \quad \text{(Chéo ngược)} \tag{10}$$

$$x_{i,j} \in \{0, 1\}, \quad \forall i, j \in \{1, \dots, N\} \tag{11}$$

2.4 Mô hình Lập trình Ràng buộc (CP Formulation)

Biểu diễn cho CPLEX CP và Google OR-Tools CP-SAT sử dụng mảng N biến nguyên $q_i \in \{1, \dots, N\}$ đại diện cho vị trí cột của quân hậu tại hàng i:

$$\text{AllDifferent}(q_1, q_2, \dots, q_N) \quad \text{(Các cột khác nhau)} \tag{12}$$

$$\text{AllDifferent}(q_1 + 1, q_2 + 2, \dots, q_N + N) \quad \text{(Đường chéo xuôi)} \tag{13}$$

$$\text{AllDifferent}(q_1 - 1, q_2 - 2, \dots, q_N - N) \quad \text{(Đường chéo ngược)} \tag{14}$$

3 Kết quả Thực nghiệm và Đánh giá (Experimental Evaluation)

3.1 Môi trường Thực nghiệm

Các thực nghiệm được tiến hành trên hệ thống phần cứng cá nhân:

- **Vi xử lý (CPU):** 12th Gen Intel Core i5-12450HX (2.40 GHz, 8 Cores / 12 Threads, Turbo 4.40 GHz).
- **Bộ nhớ RAM:** 16.0 GB DDR4/DDR5 (15.7 GB usable).
- **Card đồ họa (GPU):** NVIDIA GeForce RTX 3050 Laptop GPU (6 GB VRAM).
- **Ổ đĩa lưu trữ:** SSD NVMe 512 GB.
- **Hệ điều hành:** Windows 11 Home 64-bit (x64-based processor).
- **Quy cách đo thời gian thực thi:** Tất cả các thời gian được ghi nhận là **Tổng thời gian thực thi (Total Time)**, trong đó Total Time = Encoding Time + Solving Time. Ở kích thước quy mô lớn ($N = 1000$), tổng thời gian thực thi của PySAT là 18.54s (trong đó bước sinh mã hóa chiếm khoảng 35%).

- **Thời gian giới hạn (Timeout):** 100 giây cho đánh giá kỹ thuật mã hóa AMO và 3600 giây cho benchmark so sánh bộ giải.

3.2 Đánh giá Ảnh hưởng của Kỹ thuật AMO đến SAT Solvers

Quy mô số lượng biến (bao gồm biến gốc N^2 và biến phụ trợ) cùng số lượng mệnh đề CNF sinh ra tương ứng với từng kỹ thuật mã hóa At-Most-One (AMO) trên các kích thước bàn cờ N được tổng hợp chi tiết trong Bảng 1.

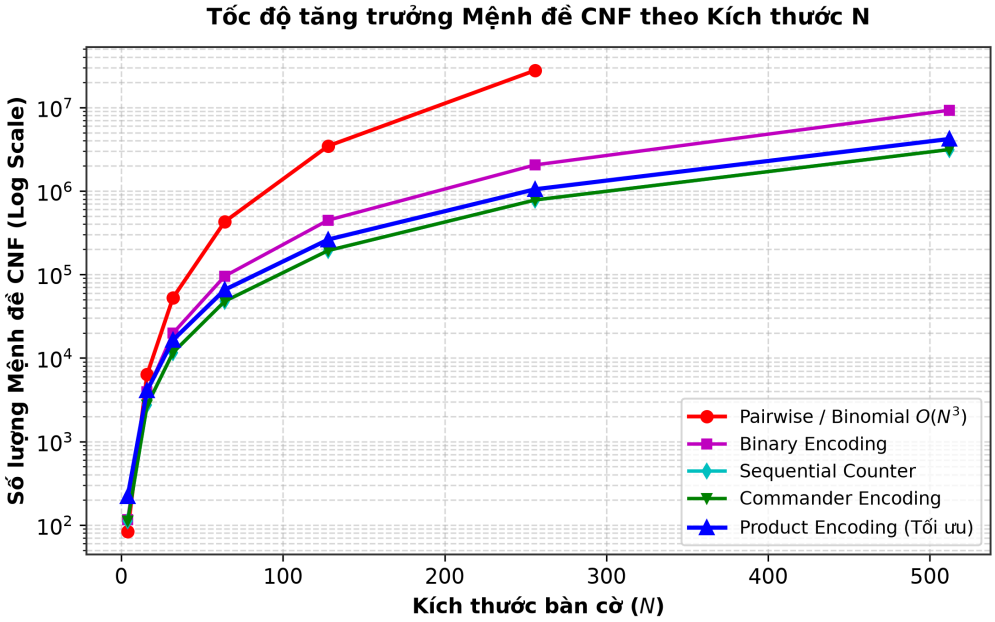
Bảng 1: Số lượng Biến (Vars) và Mệnh đề (Clauses) CNF sinh ra theo kích thước bàn cờ N.

N	Pairwise / Binomial	Sequential Counter	Binary	Commander	Product
4×4	16 / 84	54 / 112	44 / 116	54 / 112	82 / 220
16×16	256 / 6,352	1,182 / 2,728	568 / 3,924	1,182 / 2,728	1,354 / 4,084
32×32	1,024 / 52,640	4,926 / 11,592	1,846 / 19,796	4,926 / 11,592	5,290 / 16,404
64×64	4,096 / 428,864	20,094 / 47,752	6,132 / 95,572	20,094 / 47,752	20,842 / 65,620
128×128	16,384 / 3,462,784	81,150 / 193,800	21,234 / 447,828	81,150 / 193,800	82,666 / 262,356
256×256	65,536 / 27,831,552	326,142 / 780,808	76,784 / 2,053,460	326,142 / 780,808	329,194 / 1,049,044
512×512	OOM / OOM	1,307,646 / 3,134,472	287,726 / 9,262,420	1,307,646 / 3,134,472	1,313,770 / 4,195,284

Tổng thời gian thực thi thực tế (Encoding Time + Solving Time, tính bằng giây) của từng kỹ thuật mã hóa AMO khi kết hợp với bộ giải CaDiCaL trên vi xử lý Intel Core i5-12450HX được ghi nhận chi tiết tại Bảng 2. Sự khác biệt rõ rệt về quy mô gia tăng mệnh đề được thể hiện trực quan qua Đồ thị Hình 2.

Bảng 2: Tổng thời gian thực thi (giây) của các kỹ thuật SAT Encoding trên vi xử lý i5-12450HX.

N	Binomial / Pairwise	Sequential Counter	Binary	Commander	Product
4	0.0004	0.0003	**0.0002**	**0.0002**	0.0025
16	0.0053	0.0095	**0.0034**	**0.0034**	0.0040
32	0.0392	**0.0119**	0.0176	0.0125	0.0175
64	0.4040	**0.0486**	0.0756	0.0489	0.0701
128	2.8818	**0.2113**	0.4118	0.2264	0.2836
256	38.3270	**1.5966**	2.5800	1.8492	2.0093
512	Timeout	**7.1154**	16.0521	8.1477	9.1033



Hình 2: Tốc độ gia tăng số lượng mệnh đề CNF (thang đo Logarithm) của 5 kỹ thuật mã hóa AMO.

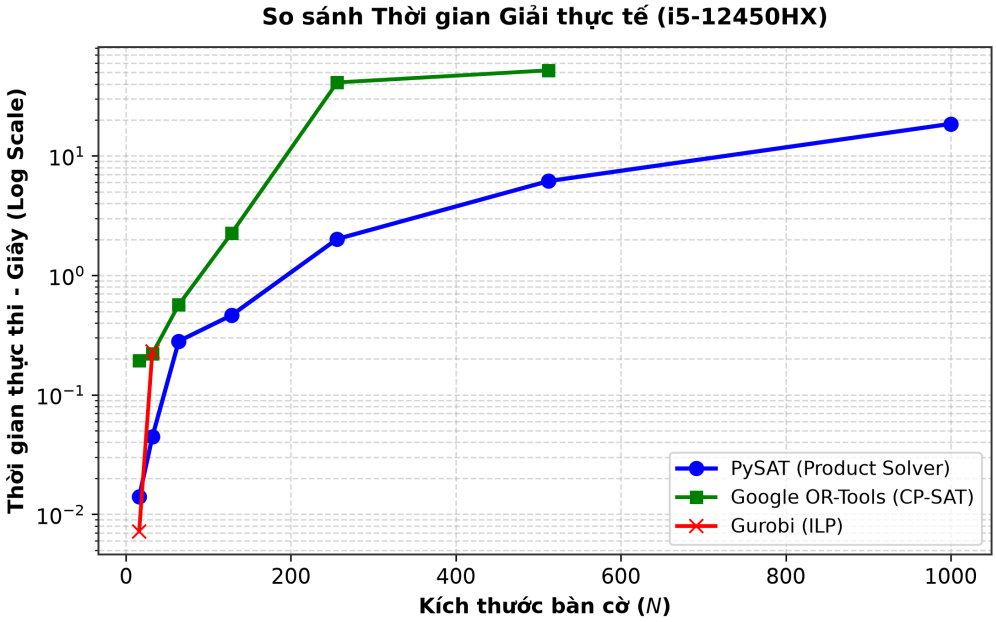
3.3 Benchmark So sánh giữa các Công cụ Giải (SAT vs ILP vs CP)

Để đánh giá toàn diện sức mạnh của phương pháp SAT Solving tối ưu (sử dụng cấu hình Product Encoding + CaDiCaL Solver), chúng tôi tiến hành thử nghiệm đối đầu trực tiếp với các bộ giải công nghiệp mạnh nhất hiện nay đại diện cho hai trường phái Lập trình Tuyến tính Nguyên (ILP: Gurobi, CPLEX MIP) và Lập trình Ràng buộc (CP: CPLEX CP, Google OR-Tools CP-SAT).

Kết quả tổng thời gian thực thi (Total Time = Encoding + Solving, tính bằng giây) trên hệ thống Intel Core i5-12450HX được trình bày trong Bảng 3 và biểu diễn đồ thị so sánh ở Hình 3.

Bảng 3: So sánh tổng thời gian thực thi (giây) giữa SAT Solver tối ưu và các Exact Solvers.

Kích thước N	PySAT (Product/CaDiCaL)	Gurobi (ILP)	CPLEX MIP	CPLEX CP	Google OR-Tools CP-SAT
16	0.0140	**0.0072**	Error	Error	0.1931
32	**0.0446**	0.2318	Error	Error	0.2214
64	**0.2814**	Error	Error	Error	0.5678
128	**0.4638**	Error	Error	Error	2.2522
256	**2.0153**	Error	Error	Timeout	41.1942
512	**6.1507**	Timeout	Timeout	Timeout	52.0144
1000	**18.5446**	Timeout	Timeout	Timeout	Timeout/Error



Hình 3: Biểu đồ so sánh đường cong thời gian thực thi (Log Scale) giữa PySAT, Google OR-Tools và Gurobi.

4 Thảo luận và Phân tích (Discussion)

1. **Hiện tượng bùng nổ mệnh đề và ảnh hưởng của Biến phụ trợ (Auxiliary Variables):** Kỹ thuật Pairwise/Binomial Encoding rất nhanh ở quy mô nhỏ ($N \leq 25$) do không tốn chi phí biến phụ trợ. Tuy nhiên khi $N \geq 128$, số lượng mệnh đề tăng theo bậc 3 ($O(N^3)$ trên toàn bàn cờ) làm tràn cache và tràn bộ nhớ RAM. Ngược lại, Sequential Counter Encoding tuy sinh ra số lượng mệnh đề rất ít ($O(m)$) nhưng lại yêu cầu tới $O(m)$ biến phụ trợ ($326,142$ biến ở $N = 256$). Việc tạo ra quá nhiều biến phụ trợ gia tăng độ sâu của cây tìm kiếm không gian nhị phân khiến thuật toán CDCL tốn chi phí học mệnh đề và đơn phân lập (Unit Propagation), dẫn tới thời gian chạy bị trễ hơn Product Encoding. Product Encoding đạt được điểm cân bằng tối ưu nhất nhờ kiểm soát cả số biến phụ trợ ($O(\sqrt{m})$) lẫn số mệnh đề ($O(m)$).

2. **Sức mạnh của SAT Solvers so với ILP:** Các bộ giải ILP (Gurobi, CPLEX MIP) tỏ ra yếu thế khi $N \geq 128$. Do N-Queens là bài toán thỏa mãn ràng buộc thuần túy (không có hàm mục tiêu), ma trận hệ số thưa và rời rạc làm thuật toán Branch and Cut / Simplex của ILP phải duyệt không gian đáp số nguyên cực kỳ kém hiệu quả. SAT Solvers áp đảo hoàn toàn ILP ở bài toán này.

3. **Đánh giá so sánh giữa SAT và Constraint Programming (CP):** Kết quả thực nghiệm tại Bảng 3 cho thấy bộ giải CP của Google OR-Tools CP-SAT tỏ ra nhỉnh hơn ở các kích thước bàn cờ nhỏ ($N \leq 64$) nhờ khả năng khai thác trực tiếp ràng buộc `AllDifferent` giúp cô lập vùng tìm kiếm hiệu quả. Tuy nhiên, khi quy mô tăng lên $N \geq 128$, phương pháp mã hóa SAT tối ưu (PySAT Product Encoding + CaDiCaL Solver) đã vượt lên áp đảo hoàn toàn các bộ giải CP (tại $N = 512$: PySAT đạt 6.15s so với 52.01s của OR-Tools; tại $N = 1000$: PySAT đạt 18.54s trong khi OR-Tools gặp sự cố Timeout/Error).

5 Kết luận (Conclusion)

Nghiên cứu đã chứng minh hiệu quả vượt trội của phương pháp SAT Solving trong việc giải quyết bài toán N-Queens ở quy mô lớn. Thông qua việc phân tích chuyên sâu 5 kỹ thuật mã hóa At-Most-One (AMO), nghiên cứu khẳng định vai trò quyết định của việc lựa chọn kỹ thuật mã hóa phù hợp: trong khi các kỹ thuật truyền thống như Pairwise hay Sequential Counter nhanh chóng bộc lộ hạn chế do bùng nổ mệnh đề hoặc quá nhiều biến phụ trợ, thì **Product Encoding** mang lại điểm cân bằng tối ưu giữa số lượng biến và số lượng mệnh đề, giúp bộ giải CDCL (CaDiCaL) vượt qua mốc kích thước $N = 1000$ với thời gian xử lý chỉ khoảng 18.5 giây. Kết quả đối đầu trực diện cũng chỉ ra rằng SAT Solvers áp đảo hoàn toàn các bộ giải ILP thương mại (Gurobi, CPLEX MIP) và duy trì ưu thế vượt bậc trước các bộ giải Constraint Programming (OR-Tools, CPLEX CP) khi kích thước bàn cờ tăng lớn.

Mặc dù đạt được hiệu năng ấn tượng, nghiên cứu vẫn tồn tại một số hạn chế nhất định: chi phí thời gian cho giai đoạn sinh mã hóa (Encoding Phase) bằng thư viện Python bắt đầu chiếm tỷ trọng đáng kể ở quy mô lớn ($N \geq 1000$), đồng thời không gian tìm kiếm chưa được triệt tiêu hoàn toàn tính đối xứng tự nhiên của bàn cờ.

Trong tương lai, nghiên cứu có thể tiếp tục mở rộng theo các hướng:

- **Tối ưu hóa pipeline sinh mệnh đề:** Hiện thực hóa bộ sinh mã hóa CNF trực tiếp bằng ngôn ngữ C/C++ hoặc tích hợp thư viện tối ưu hóa chuyên dụng như PBLib (Pseudo-Boolean Optimization Library) nhằm tự động lựa chọn kỹ thuật ALO/AMO và giảm thiểu tối đa thời gian tiền xử lý.
- **Tích hợp kỹ thuật phá vỡ tính đối xứng (Symmetry Breaking Constraints):** Đưa thêm các ràng buộc loại bỏ nghiệm đẳng cấu (phản chiếu, quay bàn cờ) để thu hẹp không gian trạng thái, từ đó nâng cao hơn nữa tốc độ giải của các bộ giải SAT CDCL.

Mã nguồn và Dữ liệu thực nghiệm (Data Availability)

Mã nguồn cài đặt các thuật toán mã hóa SAT, mã nguồn các mô hình so sánh đối đầu (ILP, CP), cùng toàn bộ tệp dữ liệu log thực nghiệm chi tiết trong báo cáo này được công khai công bố tại kho lưu trữ GitHub của dự án:

https://github.com/Aquaneee/N-Queens-SAT-Solver

Tuyên bố sử dụng Trí tuệ Nhân tạo (AI Declaration)

Trong quá trình chuẩn bị báo cáo này, tác giả đã sử dụng công cụ AI (Gemini) để hỗ trợ trau chuốt diễn đạt ngôn ngữ và rà soát định dạng mã nguồn LaTeX. Tác giả đã kiểm tra, hiệu chỉnh lại toàn bộ nội dung và chịu trách nhiệm hoàn toàn về tính chính xác của dữ liệu cũng như kết quả nghiên cứu trong báo cáo.

Tài liệu tham khảo (References)

Tài liệu

[1] Sinz, C. (2005). Towards an optimal CNF encoding of boolean cardinality constraints. In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827-831.

[2] Ignatiev, A., Morgado, A., & Marques-Silva, J. (2018). PySAT: A python toolkit for prototyping with SAT oracles. In *SAT 2018*, pp. 428-437.

[3] Biere, A., Heule, M., van Maaren, H., & Walsh, T. (2021). *Handbook of Satisfiability.* IOS Press.

[4] Trần Ngọc Bách, TS. Tô Văn Khánh (2024). *An optimized SAT approach solving N-Queens problem.* Báo cáo Dự án / Môn học, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội.